Introduction to the Theory of Computation

Lecture 10. Variants of Turing Machines and Their Equivalence

Neo (Yuan) Li

Oct 2026

Fudan University

yuan_li@fudan.edu.cn

Goal: The Turing Machine Model Is Robust

The Turing machine model is **well-defined**.

The basic model and the variants we consider have the **same computational power**:

- they recognize (or decide) the **same class of languages**;
- each can simulate the other with at most a **polynomial slowdown**: $T(n)$ steps become $\mathrm{poly}(T(n))$ steps.

Allowing the Head to Stay

Lemma 1 (stay option)
Let M be a Turing machine with transition function $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R, S\}$, where S means the head remains in place. Let $L \subseteq \Sigma^*$ be a language. If M decides L in time $T(n)$, then L is decidable in time $O(T(n))$ by a standard Turing machine.

Allowing the Head to Stay

Proof.
For each $q \in Q$, add a new state q_{back}. Keep every L/R transition, and replace each stay transition $\delta(q, a) = (q', b, S)$ by

$$\delta'(q, a) = (q'_{\text{back}},\ b,\ R),$$
$$\delta'(q'_{\text{back}}, c) = (q',\ c,\ L) \quad \text{for all } c \in \Gamma.$$

Each stay step becomes two steps, so the time is at most $2T(n)$. □

Change of Alphabet

Lemma 2 (change of alphabet)
Let $L \subseteq \{0,1\}^*$ and $T(n) \geq n^2$. If L is decided in time $T(n)$ by a Turing machine with tape alphabet Γ, then L is decided in time

$$O(\log |\Gamma| \cdot T(n))$$

by a Turing machine with tape alphabet $\{0,1,\sqcup\}$.

Proof: Change of Alphabet (1/2)

Let M be the machine on Γ, and let $k = \lceil \log_2 |\Gamma| \rceil$.

Encoding. Fix an injective map $\mathrm{enc} : \Gamma \setminus \{\sqcup\} \to \{0,1\}^k$, and let $\mathrm{enc}(\sqcup) = \sqcup^k$. Each cell of M becomes a **block** of k cells of M'.

Preprocessing. The input $x_1 x_2 \cdots x_n$ is written one bit per cell. M' first rewrites it as

$$\mathrm{enc}(x_1)\,\mathrm{enc}(x_2)\cdots\mathrm{enc}(x_n),$$

shifting the rest of the input to the right each time. This takes $O(n) \cdot O(kn) = O(kn^2)$ steps.

Proof: Change of Alphabet (2/2)

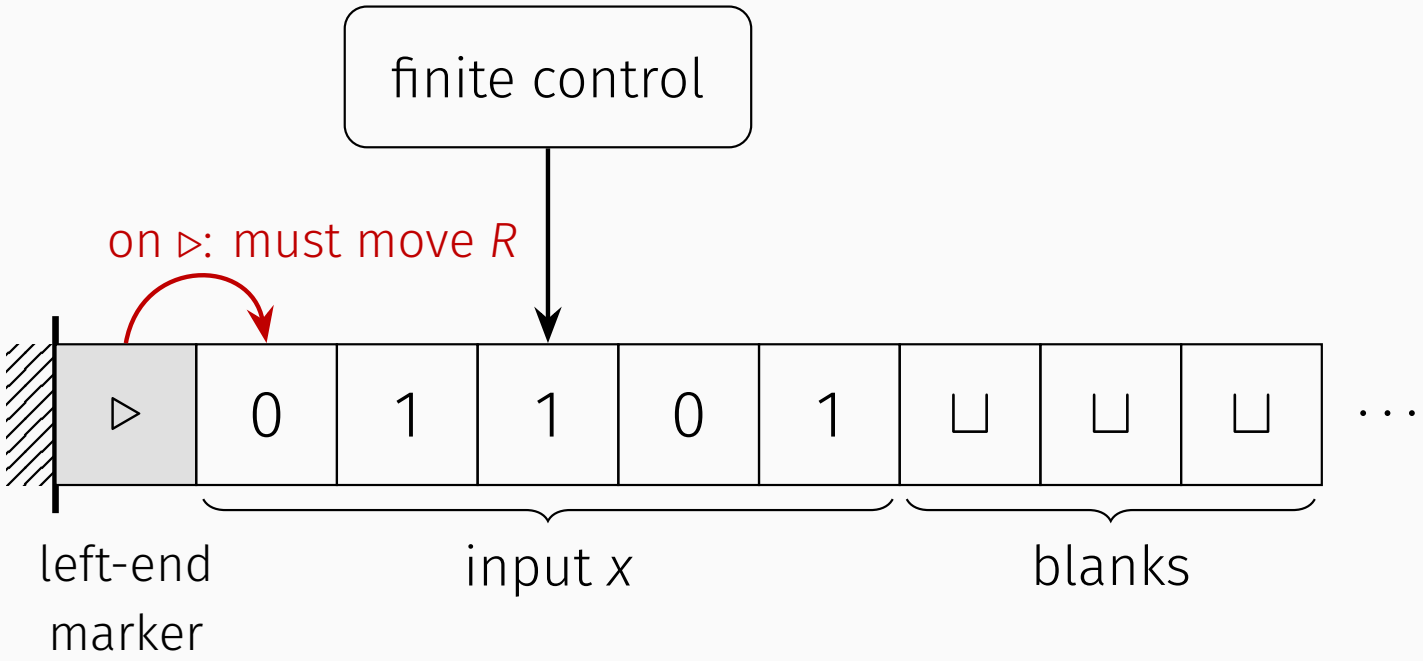
M' keeps M's state in its own state, and keeps its head on the leftmost cell of the current block. To simulate one step of M:

1. **Read:** scan the k cells of the block, remembering them in the state; this gives the symbol a. $O(k)$
2. **Decide:** compute $\delta(q, a) = (q', b, D)$ inside the state. 0
3. **Write:** scan back, writing $\mathrm{enc}(b)$. $O(k)$
4. **Move:** move k cells in direction D to the next block, and enter q'. $O(k)$

Total time: $O(kn^2) + O(k) \cdot T(n) = O(\log |\Gamma| \cdot T(n))$, since $T(n) \geq n^2$.

One-Way Turing Machines

A **one-way Turing machine** has a tape that is infinite in only one direction.



The head can never move to the left of ▷.

One-Way Turing Machines: Definition

Definition 3 (one-way Turing machine)
A *one-way Turing machine* is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where

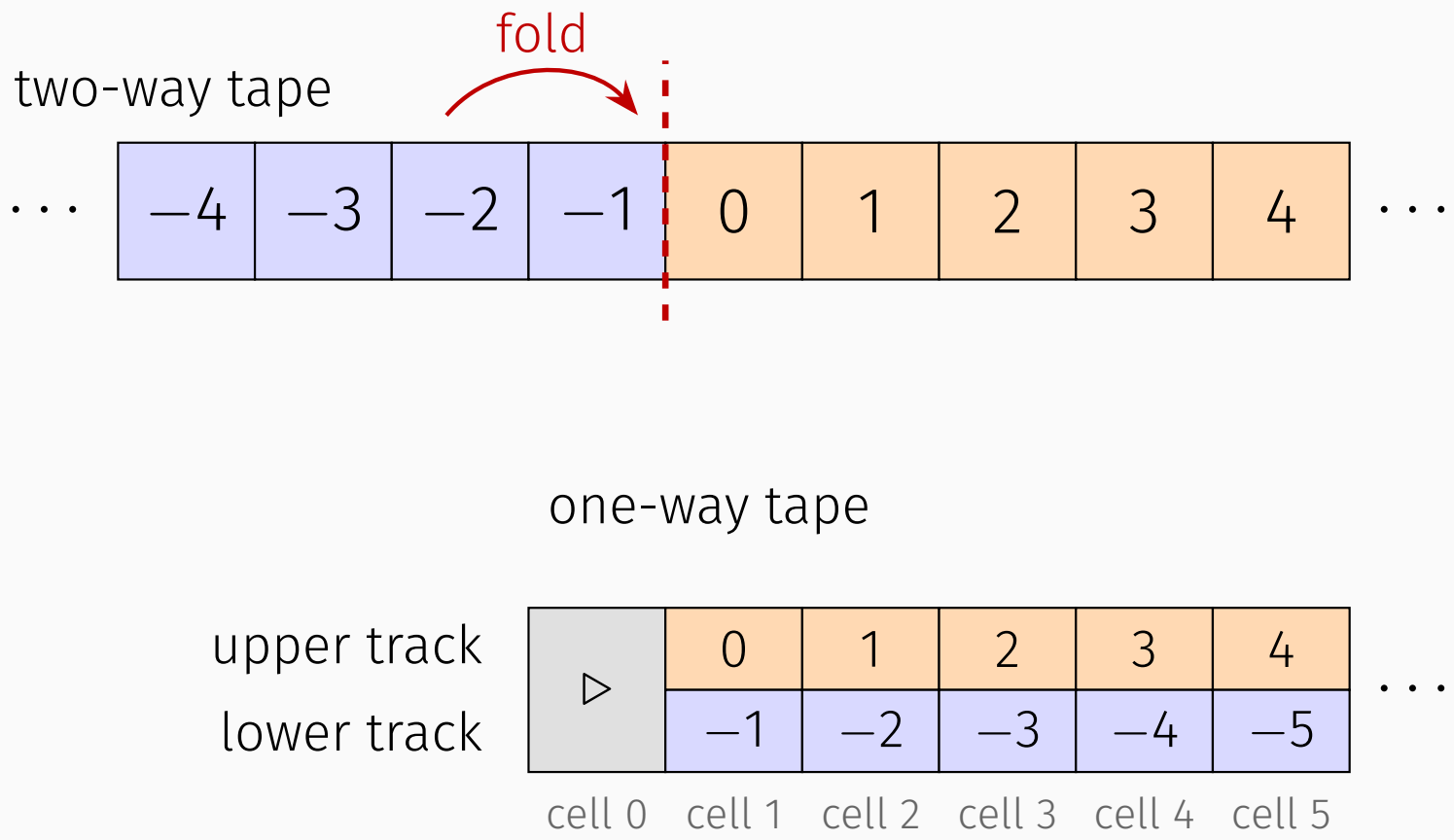
1. Q is a finite set of states;
2. Σ is the input alphabet, with $\sqcup, \triangleright \notin \Sigma$;
3. Γ is the tape alphabet, with $\Sigma \cup \{\sqcup, \triangleright\} \subseteq \Gamma$;
4. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ is the transition function, such that for every q,
$$\delta(q, \triangleright) = (q', \triangleright, R) \quad \text{for some } q' \in Q;$$
5. $q_0, q_{\text{accept}}, q_{\text{reject}} \in Q$ are the start, accept and reject states, with $q_{\text{accept}} \neq q_{\text{reject}}$.

The tape cells are indexed by $\mathbb{N}$. On input x, cell 0 holds $\triangleright$, cells $1, \ldots, |x|$ hold x, all other cells hold $\sqcup$, and the head starts on cell 1.

Simulating a Two-Way Tape

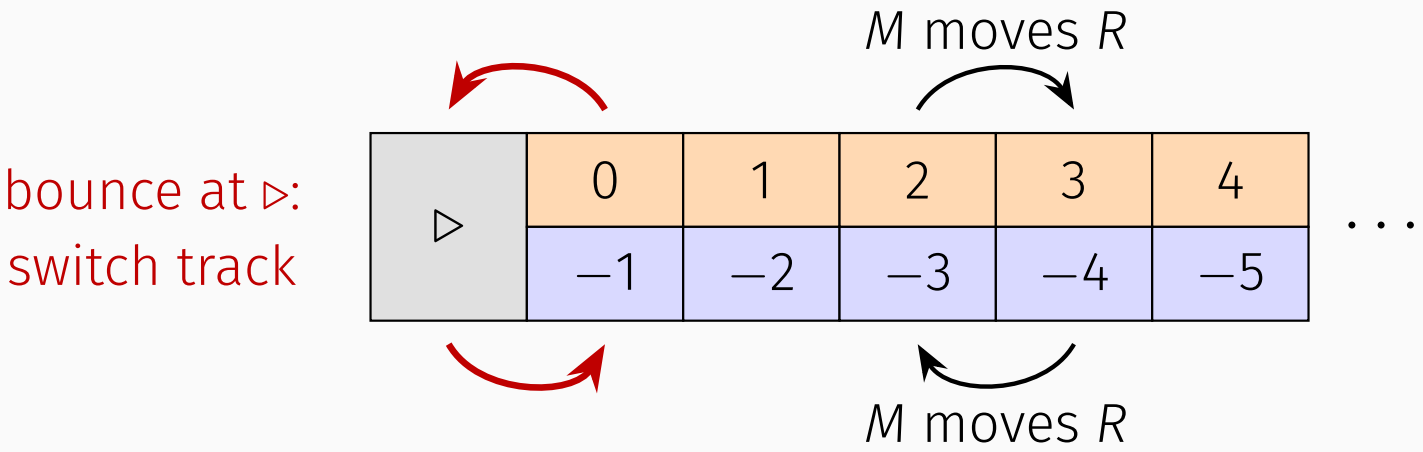
Lemma 4 (two-way vs. one-way tape)
Let $L \subseteq \{0,1\}^*$. If L is decided in time $T(n)$ by a (two-way) Turing machine, then L is decided in time $O(T(n))$ by a one-way Turing machine.

Proof Idea: Fold the Tape



Cell $j \geq 1$ of the one-way tape holds a **pair**: cell $j - 1$ (upper track) and cell $-j$ (lower track) of the two-way tape.

Proof Idea: Moving on Two Tracks



M' remembers in its state which track the head is on.

- **Upper track:** move in the **same** direction as M.
- **Lower track:** move in the **opposite** direction.
- **On ▷:** move R and switch tracks.

Proof: The Construction (1/2)

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ be the two-way machine. The one-way machine M' has:

- **tape alphabet** $\Gamma' = \Sigma \cup \{\sqcup, \triangleright\} \cup (\Gamma \times \Gamma)$;
- **states** $Q \times \{\text{up}, \text{down}\}$, starting in (q_0, up) on cell 1.

Reading a cell. The input is not rewritten in advance: a cell holding $s \in \Sigma \cup \{\sqcup\}$ is read as the pair $(s, \sqcup)$. Write $\pi(s)$ for the pair that cell s represents:

$$\pi(s) = \begin{cases} (s, \sqcup) & \text{if } s \in \Sigma \cup \{\sqcup\}, \\ s & \text{if } s \in \Gamma \times \Gamma. \end{cases}$$

Proof: The Construction (2/2)

Let $\bar{L} = R$, $\bar{R} = L$, $\overline{\text{up}} = \text{down}$, $\overline{\text{down}} = \text{up}$. For a cell s with $\pi(s) = (a, b)$:

Upper track: if $\delta(q, a) = (q', a', D)$, then

$$\delta'\big((q, \text{up}), s\big) = \big((q', \text{up}),\ (a', b),\ D\big).$$

Lower track: if $\delta(q, b) = (q', b', D)$, then

$$\delta'\big((q, \text{down}), s\big) = \big((q', \text{down}),\ (a, b'),\ \bar{D}\big).$$

Marker: $\delta'\big((q, t), \triangleright\big) = \big((q, \bar{t}),\ \triangleright,\ R\big)$.

M' accepts (rejects) as soon as M does.

Time. Each step of M costs one step of M', plus at most one extra step to bounce off $\triangleright$: total $\leq 2T(n)$.

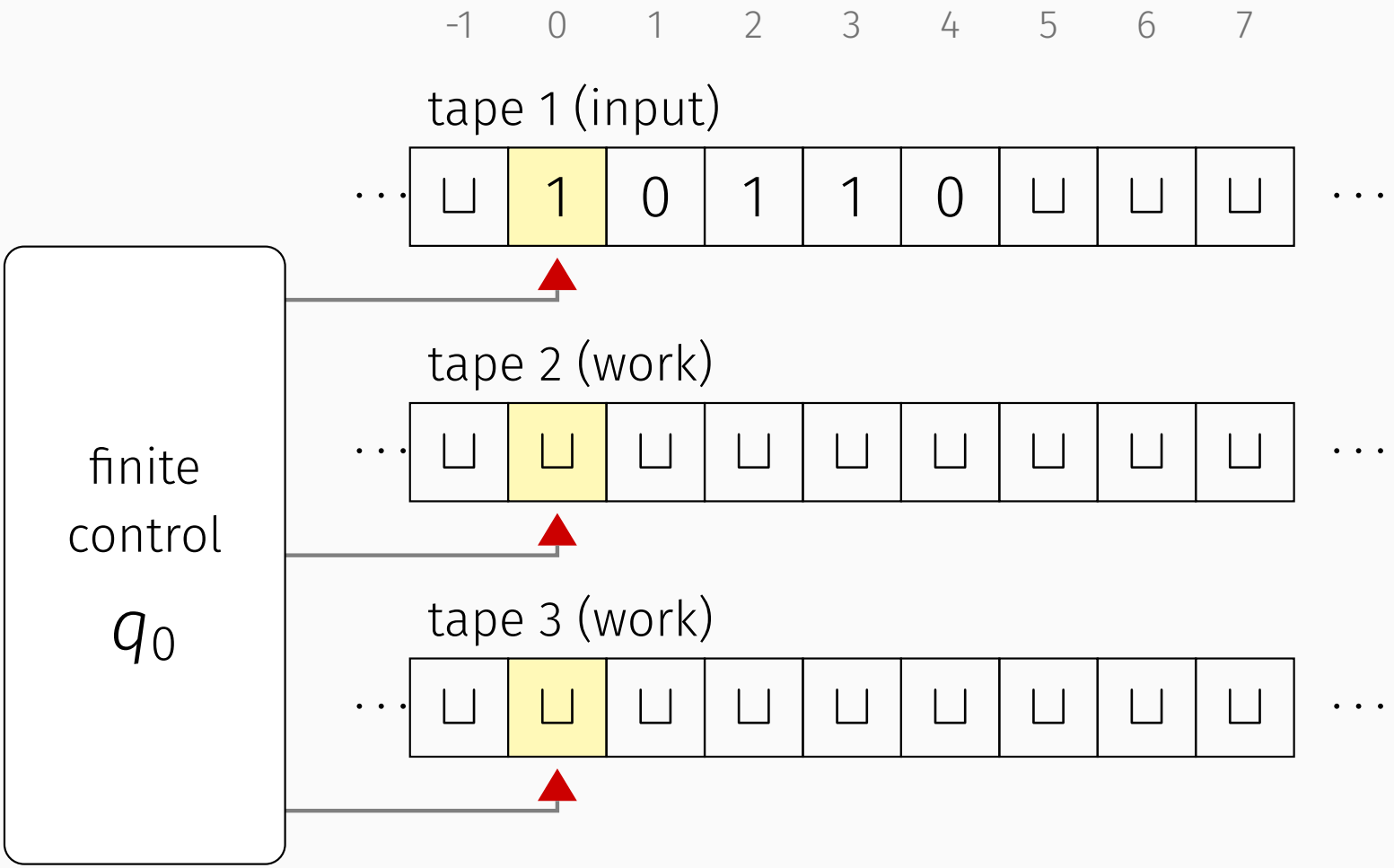
Multitape Turing Machines

Definition 5 (k-tape Turing machine)
Let $k \geq 1$ be a constant. A *k-tape Turing machine* is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, defined as for a standard Turing machine, except that it has k tapes, each with its own head, and the transition function is

$$\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{L, R, S\}^k.$$

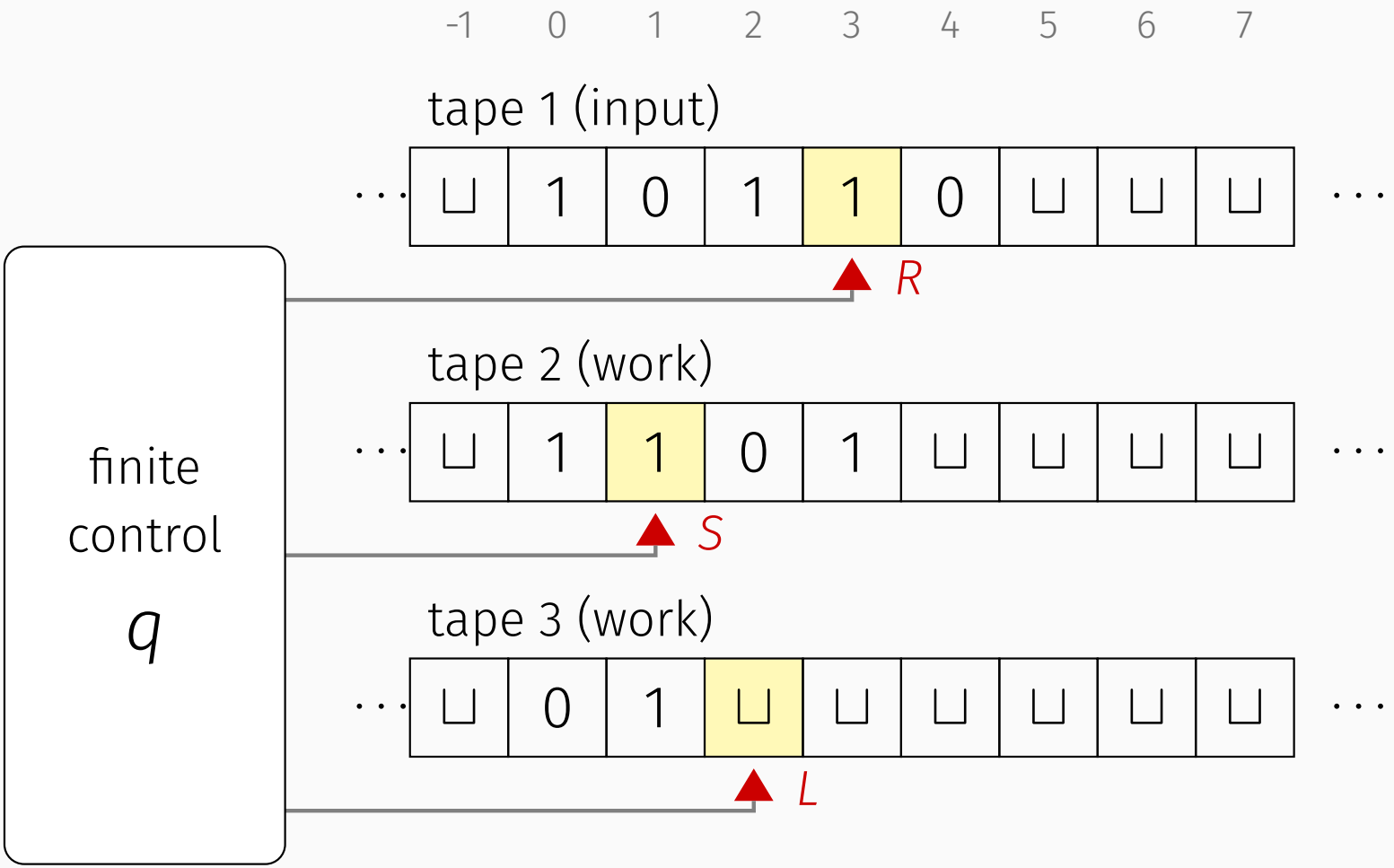
Meaning. $\delta(q, a_1, \ldots, a_k) = (q', b_1, \ldots, b_k, D_1, \ldots, D_k)$: in state q, if head i reads a_i for every i, then the machine enters q', and each head i writes b_i and moves in direction D_i.

Multitape Turing Machine: Start



- The input x is on tape 1; tapes 2 and 3 are blank.
- All three heads are on cell 0; the state is q_0.

Multitape Turing Machine: During the Computation



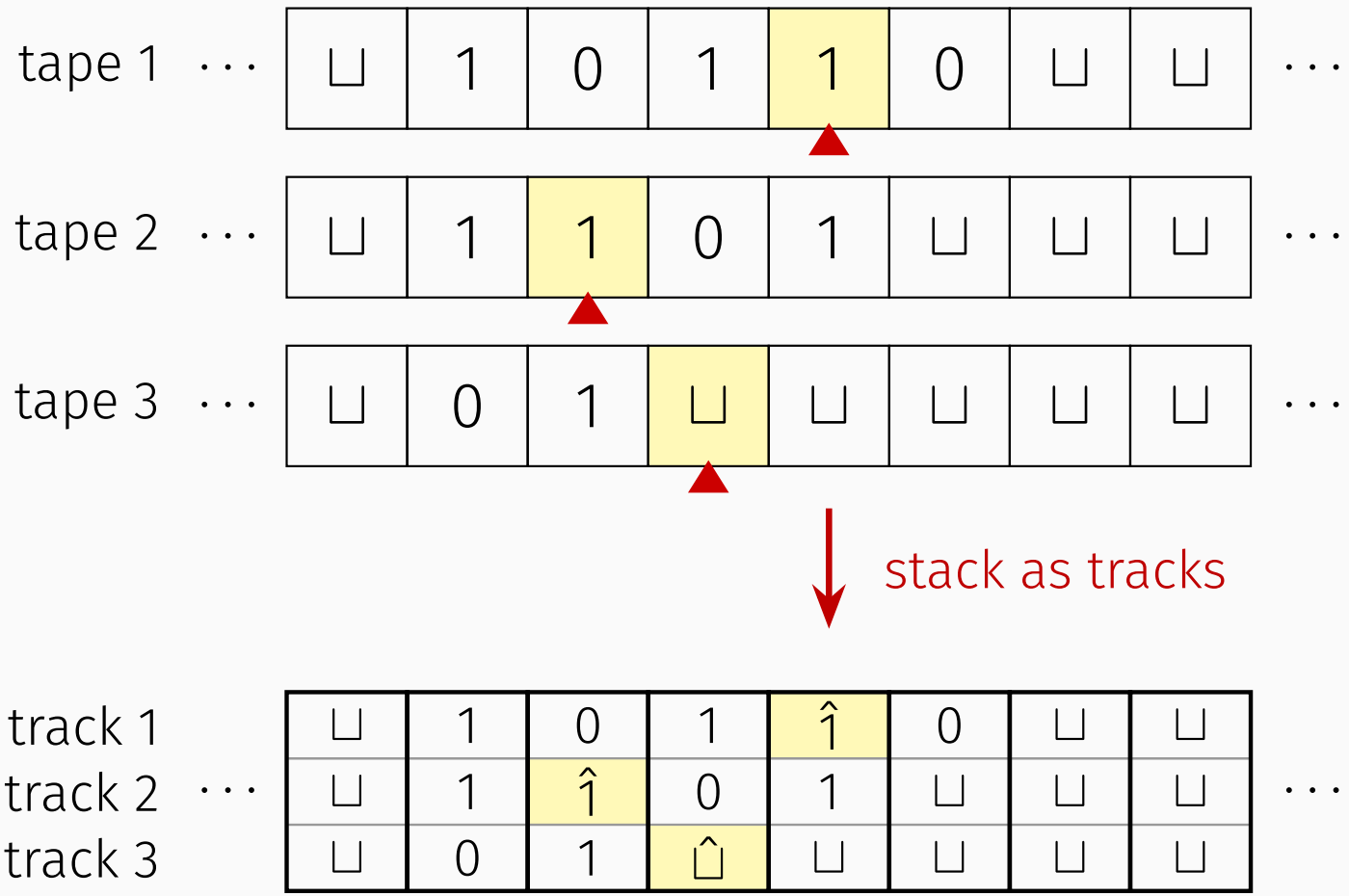
If $\delta(q, 1, 1, \sqcup) = (q', 1, 0, 1, R, S, L)$, then in **one step**: the state becomes q', the three heads write $1, 0, 1$, and move R, S, L.

Simulating a Multitape Turing Machine

Lemma 6 (multitape to single-tape)

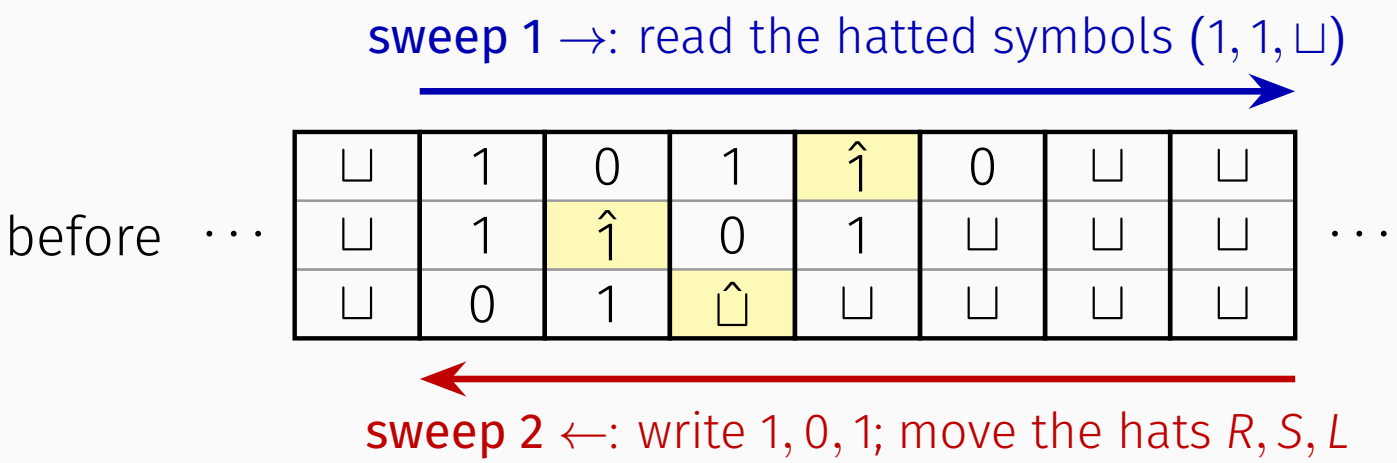
Let $k \geq 1$ be a constant, and let $T(n) \geq n$. If a language L is decided by a k-tape Turing machine in time $T(n)$, then L is decided by a single-tape Turing machine in time $O_k(T(n)^2)$.

Proof Idea: Stack the Tapes as Tracks



Each cell of the single tape holds a k-tuple. A **hat** $\hat{a}$ marks where each simulated head is.

Proof Idea: Simulating One Step



In between, M' applies $\delta(q, 1, 1, \sqcup) = (q', 1, 0, 1, R, S, L)$.
One step of M costs two sweeps.

Proof (Sketch): The Simulation

Let M be the k-tape machine. The single-tape machine M':

- **Tracks.** Each cell holds a k-tuple of symbols of Γ, each possibly hatted. Hats mark the k head positions. An input cell holding x_j is read as $(x_j, \sqcup, \ldots, \sqcup)$.
- **Start.** Put all k hats on cell 0.
- **One step of M.**
  1. Sweep right over the used cells; remember the k hatted symbols in the state.
  2. Compute δ of M inside the state.
  3. Sweep left; write the new symbols and move each hat by at most one cell.
- **Halt** with M's answer as soon as M halts.

Proof (Sketch): Running Time

- In $T(n)$ steps, every head of M stays within cells $-T(n), \ldots, T(n)$.
- So the used part of the tape of M' has at most $2T(n) + 1$ cells.
- Each step of M costs two sweeps, plus $O(k)$ extra moves: $O(T(n))$ steps of M'.

Total time:

$$T(n) \cdot O(T(n) + k) = O_k(T(n)^2).$$

□

Definition: DTIME

Definition 7 (DTIME)
Let $T : \mathbb{N} \to \mathbb{N}$ be a function. A language $L \subseteq \{0,1\}^*$ is in $\mathrm{DTIME}(T(n))$ if there exists a multitape Turing machine M that decides L in time $O(T(n))$.

Definition: P

Definition 8

$$\begin{aligned} \mathrm{P} &= \bigcup_{c \geq 1} \mathrm{DTIME}(n^c) \\ &= \mathrm{DTIME}(n) \cup \mathrm{DTIME}(n^2) \cup \mathrm{DTIME}(n^3) \cup \cdots \end{aligned}$$

Remark. P does not depend on the choice of Turing machine model, since these models can all simulate each other in polynomial time.

Two Conjectures

$$\mathrm{CLIQUE} = \{\, \langle G, k \rangle : G \text{ has a clique of size } k \,\}.$$

Conjecture 9

$\mathrm{P} \neq \mathrm{NP}$. *Equivalently,* $\mathrm{CLIQUE} \notin \mathrm{P}$.

Conjecture 10

$\mathrm{CLIQUE} \notin \mathrm{DTIME}(n)$.

Definition: RAM Turing Machine

Definition 11
A *RAM Turing machine* is a $(k+1)$-tape Turing machine

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}, Q_{\text{access}})$$

with the following extra features:

- tapes $1, \ldots, k$ are work tapes (tape 1 holds the input); tape $k+1$ is the *address tape*;
- a *memory* $A : \mathbb{N} \to \Gamma$, initially $A[i] = \sqcup$ for all i;
- a set of *access states* $Q_{\text{access}} \subseteq Q \setminus \{q_{\text{accept}}, q_{\text{reject}}\}$.

Definition: Memory Access

Γ contains two special symbols Rd and Wr. When M enters a state in Q_{access}:

- **Read.** If the address tape contains $\mathrm{bin}(i)\,\mathrm{Rd}$, then the address tape is erased and $A[i]$ is written into its first cell.
- **Write.** If the address tape contains $\mathrm{bin}(i)\,\mathrm{Wr}\,\sigma$, then $A[i]$ is set to σ and the address tape is erased.
- Otherwise, nothing happens.

Each access counts as one step.

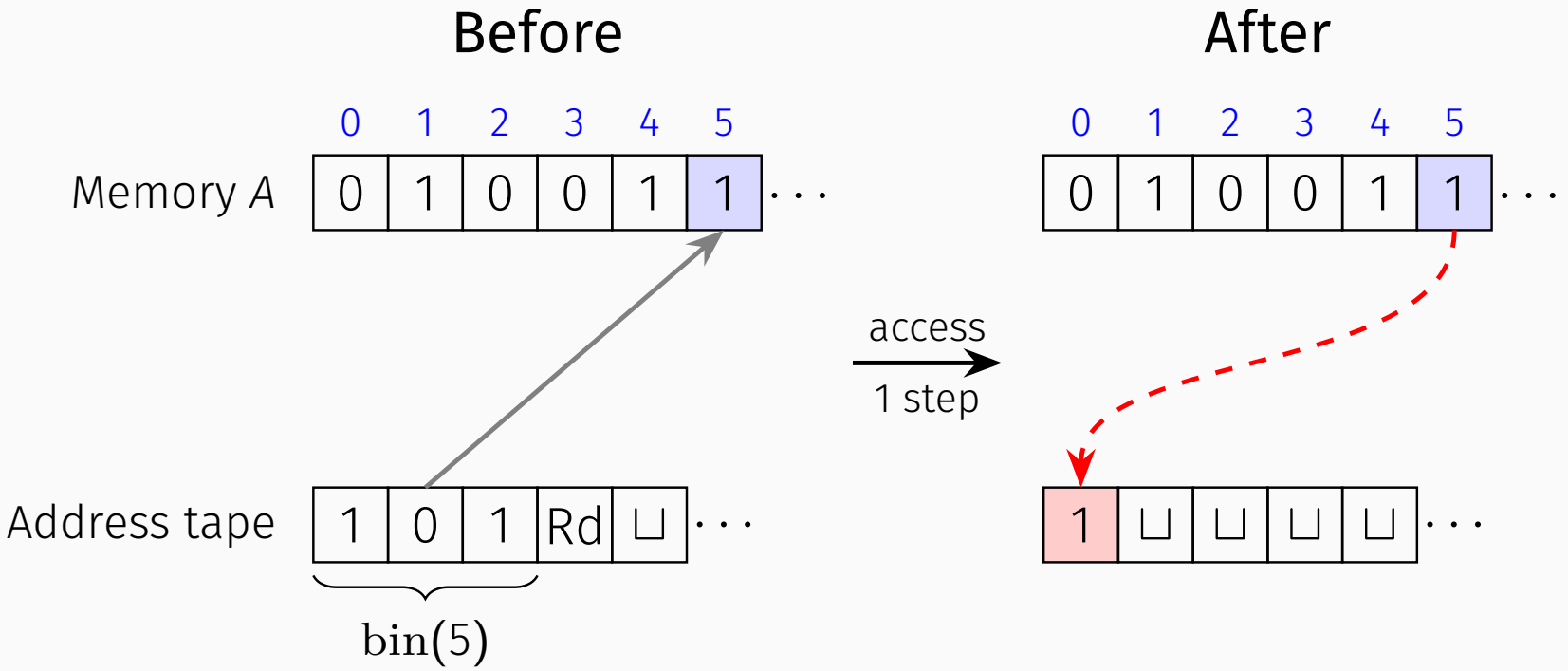
Definition: Transition Function

The transition function is that of an ordinary $(k+1)$-tape machine:

$$\delta : Q \times \Gamma^{k+1} \to Q \times \Gamma^{k+1} \times \{\mathrm{L}, \mathrm{S}, \mathrm{R}\}^{k+1}.$$

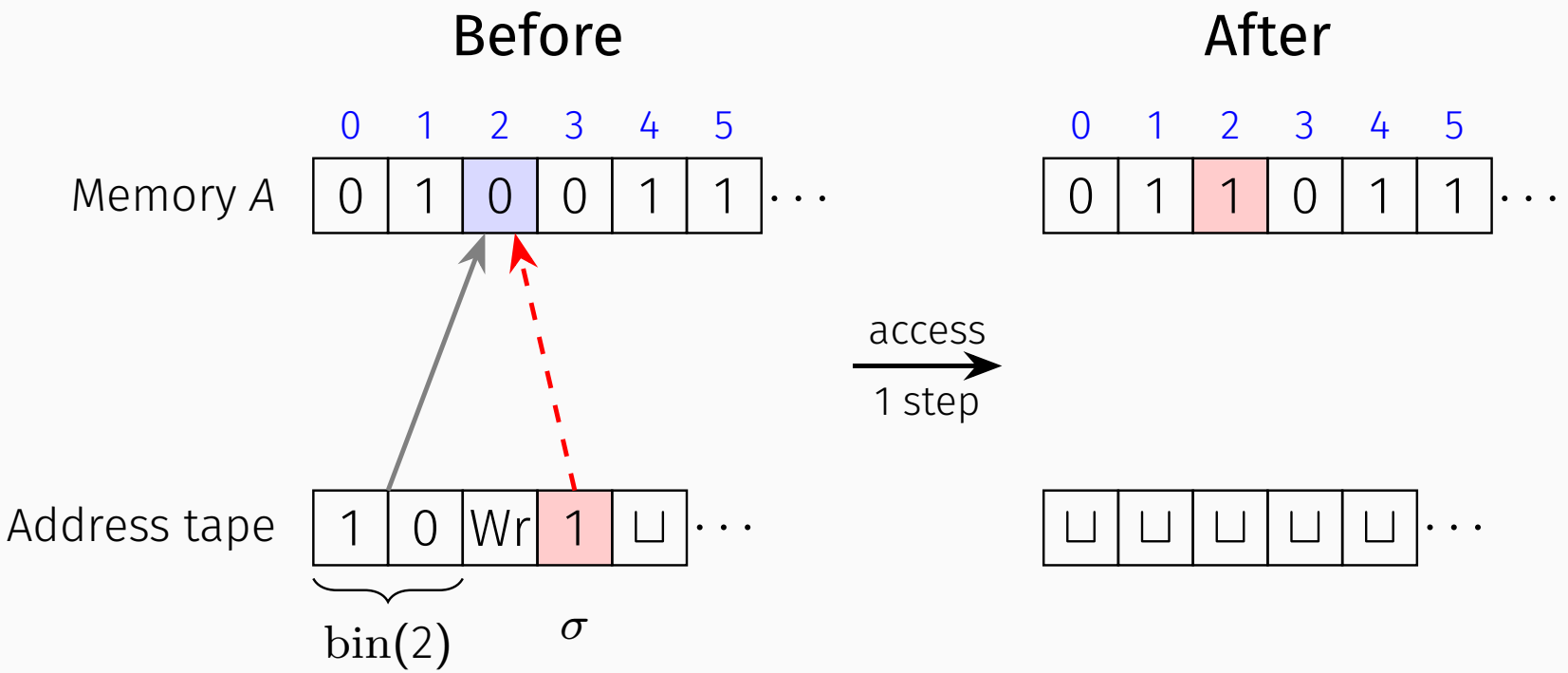
The memory A does not appear in δ; it is used only through access states.

RAM TM Example: Read



The address tape holds $\mathrm{bin}(5)$ Rd (gray arrow: the address). After one access step, $A[5] = 1$ is copied into the first cell of the address tape (red), and the rest is erased.

RAM TM Example: Write



The address tape holds $\mathrm{bin}(2)\,\mathrm{Wr}\,1$ (gray arrow: the address; red arrow: the value σ). After one access step, $A[2]$ changes from 0 to 1, and the address tape is erased.

Lemma: Simulating a RAM Turing Machine

Lemma 12

If a language $L \subseteq \Sigma^*$ is decidable by a RAM TM in time $T(n)$, then L is decidable by a multitape TM in time $O(T(n)^2)$.

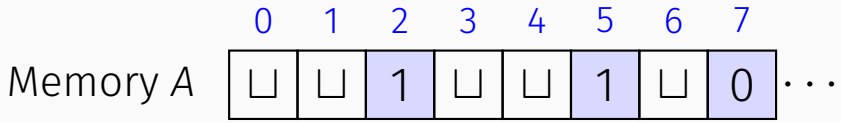
Proof: Idea

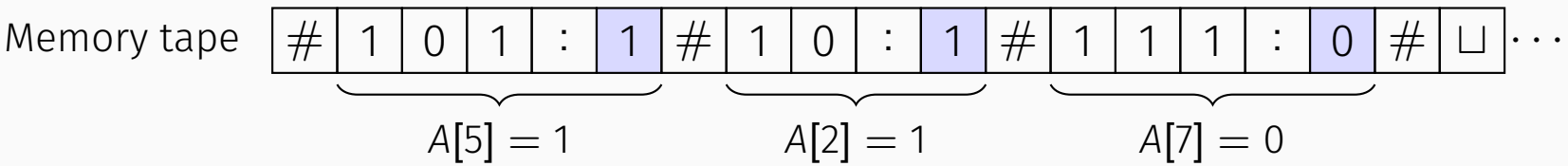
Let M be a RAM TM running in time $T(n)$. The multitape TM M':

- keeps M's k work tapes and address tape as ordinary tapes;
- adds one more tape, the *memory tape*, holding a list of pairs $(\mathrm{bin}(i), A[i])$, one for each address i accessed so far;
- simulates each non-access step of M in one step;
- simulates each access by scanning the memory tape.

Addresses never accessed are not stored; their value is $\sqcup$.

Proof: The Memory Tape





Addresses $5, 2, 7$ have been accessed (in this order). Each is stored as $\#\,\mathrm{bin}(i) : A[i]$. All other cells of A are implicitly ⊔.

Proof: Simulating One Access

When M enters an access state, M' does the following.

1. Scan the memory tape from left to right. For each entry, compare its address with the address tape symbol by symbol, stopping at the first mismatch.
2. **Read** $\mathrm{bin}(i)\,\mathrm{Rd}$: let a be the value stored for i ($a = \sqcup$ if there is no entry). Erase the address tape and write a into its first cell.
3. **Write** $\mathrm{bin}(i)\,\mathrm{Wr}\,\sigma$: overwrite the value of the entry for i with σ, or append a new entry $\#\,\mathrm{bin}(i) : \sigma$ if there is none. Erase the address tape.
4. Move the memory tape head back to the left end.

Proof: Addresses Are Short in Total

Let $i_1, i_2, \ldots, i_m$ be the addresses accessed by M, and $\ell_j = |\mathrm{bin}(i_j)|$.

- Right after an access, the address tape holds at most one non-blank symbol.
- Before access j, it must hold $\mathrm{bin}(i_j)$ followed by Rd or Wr, so M must write at least ℓ_j symbols.
- Each step writes at most one symbol on the address tape.

Hence

$$\ell_1 + \ell_2 + \cdots + \ell_m \leq T(n),$$

and the memory tape has length $O(T(n))$.

Proof: Running Time

- A non-access step of M costs $O(1)$ steps of M'.
- An access costs $O(T(n))$ steps of M':
  - comparing with one entry, including moving the address tape head back, costs $O(\text{length of the entry})$, so one scan costs $O(\text{length of the memory tape}) = O(T(n))$;
  - erasing the address tape, appending an entry, and moving the memory head back each cost $O(T(n))$.

Total time:

$$T(n) \cdot O(T(n)) = O(T(n)^2).$$

Summary: The Turing Machine Model Is Robust

Variant	Simulated in time	Idea
stay option (S)	$O(T(n))$	stay = right, then left
alphabet Γ	$O(\log \lvert\Gamma\rvert \cdot T(n))$ *	encode each symbol in $\lceil \log_2 \lvert\Gamma\rvert \rceil$ bits
two-way tape	$O(T(n))$	fold the tape into two tracks
k tapes	$O_k(T(n)^2)$	k tracks, hats mark heads
RAM TM	$O(T(n)^2)$ †	list of (address, value) pairs

* assuming $T(n) \geq n^2$. † on a multitape TM; $O(T(n)^4)$ on a single-tape TM.

Consequence. Every variant changes $T(n)$ to at most $\mathrm{poly}(T(n))$, so

- the class of decidable languages is the same;
- P is the same, whichever model we use.